O sistema de garagem foi desenvolvido em TypeScript e tem como objetivo gerenciar veículos, permitindo adicionar, remover, consultar e localizar carros.

O projeto é composto por três arquivos: `Car.ts`, `Garage.ts` e `garagem.jest.ts`. Os testes automatizados são realizados utilizando o framework Jest.

Estrutura: Car.ts

A classe `Car` representa um veículo e possui os seguintes atributos privados:

- `brand`: marca do veículo.
- `model`: modelo do veículo.
- `year`: ano do veículo.
- `stored`: estado de armazenamento.

O construtor inicializa os dados do carro e define `stored` como `false`.

Os métodos disponíveis são:

- `getBrand()`: retorna a marca.
- `getModel()`: retorna o modelo.
- `getYear()`: retorna o ano.
- `setStored()`: altera o estado de armazenamento.
- `isStored()`: consulta o estado de armazenamento.

Garage.ts

A classe `Garage` gerencia uma lista de objetos `Car`, armazenada no atributo privado `cars`.

Seus principais métodos são:

- `addCar(car)`: adiciona um veículo, verificando duplicidade e estado de armazenamento.
- `removeCar(car)`: remove um veículo existente e atualiza seu estado.
- `getCarCount()`: retorna a quantidade de carros armazenados.
- `findCarByModel(model)`: localiza um veículo pelo modelo.

A classe também implementa regras para impedir a inclusão de veículos duplicados ou já marcados como armazenados, além de impedir a remoção de objetos que não estejam na lista.

garagem.jest.ts

O arquivo contém os testes automatizados da classe `Car`, utilizando `describe()`, `test()`, `expect()` e `toBe()`.

Os cenários implementados verificam a criação do veículo, a consulta dos dados e o estado inicial de armazenamento. O terceiro teste chama `setStored(true)`, mas precisa de uma asserção adicional para confirmar a alteração.

Casos de teste

ID	Caso de teste	Resultado esperado
CT01	Criar um carro	Dados do veículo retornados corretamente.
CT02	Verificar estado inicial	`stored` igual a `false`
CT03	Alterar estado de armazenamento	`isStored()` retorna `true`.
CT04	Inicializar garagem	Quantidade inicial igual a `0`
CT05	Adicionar carro	Veículo adicionado e estado igual a `true`.
CT06	Adicionar carro duplicado	Operação rejeitada com erro.
CT07	Adicionar carro já armazenado	Operação rejeitada com erro.
CT08	Remover carro	Veículo removido e estado igual a `false`.
CT09	Remover carro inexistente	Operação rejeitada com erro.
CT10	Contar veículos	Quantidade retornada corretamente.
CT11	Buscar modelo existente	Objeto correspondente retornado.
CT12	Buscar modelo inexistente	Retorno igual a `undefined`.

Os casos CT01 a CT03 correspondem aos testes presentes no arquivo fornecido. Os demais são testes recomendados para ampliar a cobertura da classe `Garage`.

Resultados dos testes

Os resultados devem ser registrados após a execução dos testes automatizados.

- **Passou:** resultado obtido corresponde ao esperado.
- **Falhou:** resultado obtido diverge do esperado.
- **Pendente:** teste ainda não executado ou validação incompleta.

Não foram registrados resultados reais nesta documentação, pois a execução dos testes não foi apresentada.